

Part 02

2. Define an enum called `DayOfWeek` with values: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday.

Write a program that takes an integer input from the user (1-7) and prints the corresponding day using the enum.

Use `Enum.Parse` to convert an integer to an enum value.

```
using System;

enum DayOfWeek
{
    Monday = 1,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday,
    Sunday
}

class Program
{
    static void Main()
    {
        Console.Write("Enter a number (1-7): ");
        int input = int.Parse(Console.ReadLine());

        DayOfWeek day = (DayOfWeek)Enum.Parse(typeof(DayOfWeek), input.ToString());

        Console.WriteLine($"The day is: {day}");
    }
}
```

Explanation

The `DayOfWeek` enum contains seven values, starting from `Monday = 1` and ending with `Sunday = 7`.

The program takes an integer from the user, converts it to a string using `ToString()`, and then uses `Enum.Parse()` to convert it to the corresponding `DayOfWeek` enum value.

For example, if the user enters 3, the program prints:

The corresponding day is: Wednesday

3- What happens if the user enters a value outside the range of 1 to 7?

Answer

If the user enters a value outside the range of **1 to 7**, the program will not find a corresponding `DayOfWeek` value. Therefore, the input should be validated before converting it to the enum to prevent invalid input.

For example:

```
if (input < 1 || input > 7)
{
    Console.WriteLine("Invalid input. Please enter a number between 1 and 7.");
}
else
{
    DayOfWeek day = (DayOfWeek)Enum.Parse(
        typeof(DayOfWeek),
        input.ToString()
    );

    Console.WriteLine($"The corresponding day is: {day}");
}
```

This validation ensures that only values from **1 to 7** are accepted.